EE656 – Artificial Intelligence, Machine Learning, Deep Learning and Its Applications

Intelligent Condition-Based Monitoring of Air Compressors Using Acoustic Signals

An implementation and analysis of the methodology by N. K. Verma et al., IEEE Transactions on Reliability (2016)

Project Report

Team 1

Bala Swetha S (240256)
Boddupally Utham Teja Sree (240280)
Eragandula Nehasri (240384)
Yash Jaiswal (241196)
Yashjeet Singh (241208)

Instructor: Prof. Nishchal K. Verma

Department of Electrical Engineering, IIT Kanpur

Semester: 2025–26 (Summer Term)

Abstract

In this project we implemented and analysed a complete acoustic-signal-based fault diagnosis system for reciprocating air compressors, following the framework laid out by Verma et al. (2016). Our goal was not simply to reproduce a number, but to understand *why* each stage of the pipeline exists and what it contributes. We worked with acoustic recordings of a single-stage air compressor running in eight states - one healthy and seven faulty (valve leakages, piston-ring, flywheel, rider-belt, and bearing faults) and built every stage of the pipeline ourselves in Python.

The pipeline comprises sensitive position analysis (SPA) using Empirical Mode Decomposition (EMD) and the Hilbert Transform; a four-step pre-processing routine (filtering, clipping, smoothing, and a robust max–min normalisation); multi-domain feature extraction across time, frequency, and time–frequency domains yielding 286 base features (extended to 629 with additional transforms); six dimensionality-reduction and feature-selection methods (PCA, LDA, MIFS, mRMR, NMIFS, and Bhattacharyya Distance); and multiclass classification using several models, with a Support Vector Classifier as our primary choice. Evaluating across the One-Against-One, One-Against-All, and DDAG multiclass strategies with k-fold cross-validation, our best configuration (mRMR-selected features, OAA decomposition) reached **99.44%** classification accuracy, and most well-chosen configurations stayed at or above 99%. The key lesson we took away is that careful, domain-informed feature engineering matters more than classifier complexity or an exotic feature budget: a classical SVM on a compact set of well-chosen acoustic features rivals a deep CNN on spectrograms, at a fraction of the training and inference cost, and a 629-feature "extended" transform set buys almost nothing over the original 286 once a good selector is in the loop.

Abbreviations

AE	Acoustic Emission
BD	Bhattacharyya Distance
BJD	Born–Jordan Distribution
CBM	Condition-Based Maintenance
CWD	Choi–Williams Distribution
DAQ	Data Acquisition
db4	Daubechies-4 wavelet
DCT	Discrete Cosine Transform
DDAG	Decision Directed Acyclic Graph
DFT	Discrete Fourier Transform
DWT	Discrete Wavelet Transform
EMD	Empirical Mode Decomposition
FFT	Fast Fourier Transform
FD	Fault Detection
IMF	Intrinsic Mode Function
KNN	K-Nearest Neighbours
RF	Random Forest
RBF	Radial Basis Function
UMT	Updated Morlet Transform
WVD	Wigner-Ville Distribution
TD	Time Domain
LDA	Linear Discriminant Analysis
LIV	Leakage Inlet Valve
LOV	Leakage Outlet Valve
MI	Mutual Information
MIFS	Mutual Information Feature Selection
MLP	Multi-Layer Perceptron
mRMR	Min. Redundancy Max. Relevance
MWT	Morlet Wavelet Transform
NMIFS	Normalised MIFS
NRV	Non-Return Valve
OAA	One-Against-All
OAO	One-Against-One
PCA	Principal Component Analysis
RMS	Root Mean Square
SPA	Sensitive Position Analysis
STFT	Short-Time Fourier Transform
SVC	Support Vector Classifier
SVM	Support Vector Machine
WPT	Wavelet Packet Transform
PWVD	Pseudo Wigner-Ville Distribution
AC	Autocorrelation

Contents

1. Introduction

1.1 Why This Problem Matters

Modern industry runs on rotating machinery such as pumps, conveyors, motors, and air compressors that operate continuously, often in harsh conditions. When one of these machines fails unexpectedly, the cost is rarely just the price of a replacement part. It is the lost production while the line is down, the cascading damage to neighbouring components, and in the worst cases, the safety risk to the people working nearby. For reciprocating air compressors in particular, the valves, piston rings, and bearings endure constant mechanical stress and are the usual first points of failure.

The traditional way of catching these faults early is to have an experienced technician periodically inspect the machine that is listening to it, feeling for vibration, checking pressures. This works, but it does not scale, it is subjective, and it depends entirely on the availability of a skilled human. The modern alternative is **condition-based maintenance (CBM)**: instrument the machine, continuously monitor a measurable signal, and let an algorithm flag the early signature of a developing fault. Of the three classic maintenance philosophies - run-to-failure (breakdown), fixed-schedule (preventive), and condition-based, CBM is widely regarded as the most cost-effective because it triggers maintenance *only* when the data say it is needed.

1.2 Why Acoustic Signals?

A CBM system can monitor many physical quantities: vibration, temperature, pressure, motor current, or sound. We chose to study an *acoustic* system for a few concrete reasons that make it especially attractive for a course project and for real deployment alike:

- **Non-contact sensing.** A microphone does not need to be bolted onto the machine the way an accelerometer does. It can sit nearby and still pick up fault signatures, which simplifies installation and lets one sensor watch several components at once.
- **Early detection.** Acoustic emission tends to reveal incipient faults - the very first signs of wear earlier than heat or noticeable noise does. Figure 1 (conceptual) shows how acoustic and vibration methods catch problems well before thermal methods would.
- **Robustness.** Acoustic signals are relatively insensitive to structural resonances and mechanical background noise compared to raw vibration, giving cleaner fault signatures.
- **Low cost.** Microphones and a sound card are cheap. The intelligence lives in software, which is exactly where machine learning shines.

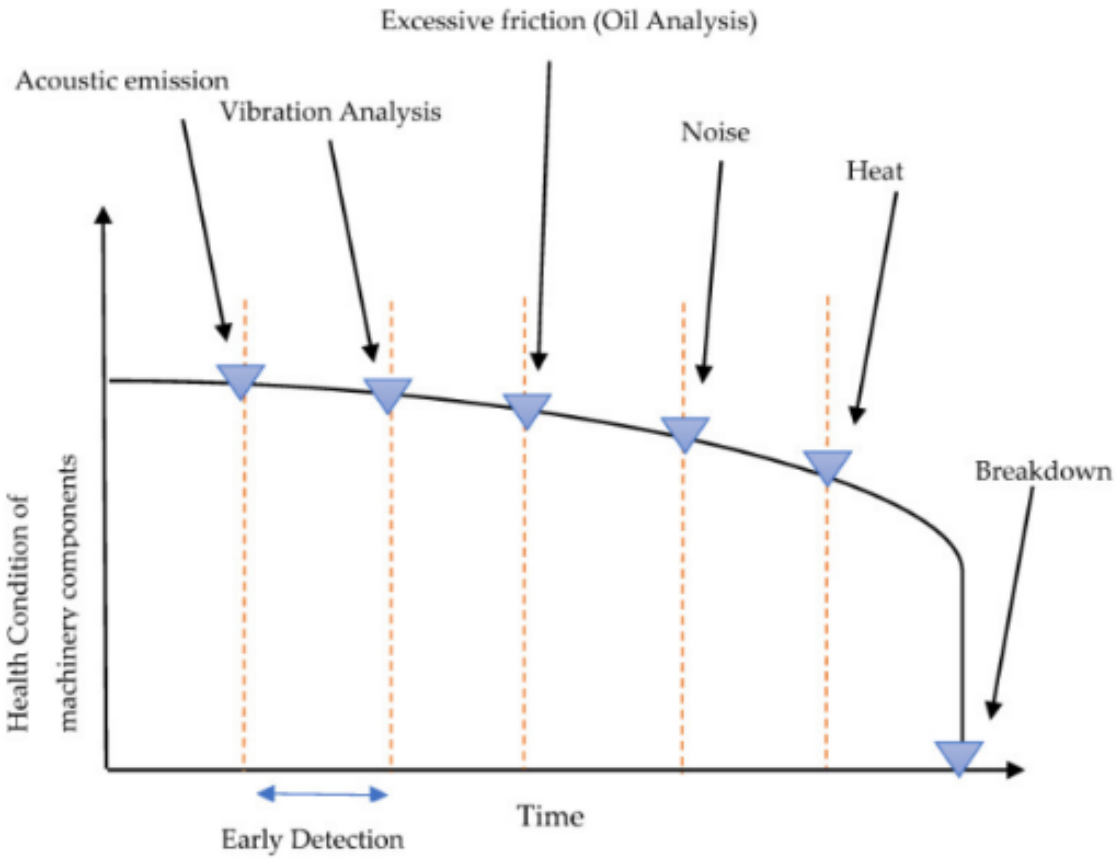

Figure 1: Detection sensitivity of acoustic and vibration methods relative to other techniques across the degradation timeline of a machine component.

1.3 What We Set Out To Do

We framed our project around a small set of questions that we wanted to answer through our own experiments rather than take on faith from the paper:

Q1. Can we reproduce the paper's headline claim which is better than 99% accuracy from a *single* microphone by building the full pipeline ourselves?

Q2. Which feature-selection method actually works best in practice, and how few features can we get away with before accuracy collapses?

Q3. Which signal domains (time, frequency, or time–frequency) carry the most fault information?

Q4. How much does the choice of classifier and the choice of multiclass strategy (OAO vs. OAA vs. DDAG) really matter for accuracy *and* for compute cost?

Q5. Does enlarging the feature set with more exotic transforms actually buy us anything once a good selector is already in the loop?

Q6. Is this classical pipeline competitive with a deep learning approach, given that deep learning is the obvious modern alternative?

1.4 Scope and Report Structure

We restrict ourselves to classifying eight discrete machine states from acoustic recordings taken at a single, well-chosen sensor location. We do not attempt to detect multiple simultaneous faults, nor do we do severity estimation. The rest of the report is organised as follows. Section 2 gives the background and the standard CBM pipeline. Section 3 covers data acquisition and our sensitive position analysis. Section 4 details pre-processing. Section 5 describes feature extraction with the formulas we implemented. Section 6 covers feature selection and dimensionality reduction. Section 7 presents our classification models, evaluation methodology, and the full set of experimental results. Sections 9–10 reflect on what we learned and where this could go next.

2. Background and Related Work

2.1 From Manual Inspection to Data-Driven Diagnosis

Traditional fault detection leans on periodic manual inspection - a technician listening for unusual sounds, feeling for vibration, or checking gauges. This works for a handful of machines, but it scales poorly: it is labour-intensive, slow, and inconsistent because it depends on the judgement and availability of a skilled human. As industrial systems grew larger and more instrumented, the field shifted decisively toward **intelligent, data-driven** fault detection, in which sensors stream signals continuously and a machine-learning model flags the early signature of a developing fault. The appeal is that the model never tires, never forgets, and can watch many machines at once.

3. The Dataset

3.1 Sensor-Based Fault Detection

Two sensor families dominate fault detection in rotating machinery. **Vibration sensors** (accelerometers) give direct, highly sensitive measurements but require physical contact and careful mounting, and each one typically watches a single point. **Acoustic sensors** (microphones) trade a little sensitivity for the practical advantages described above: non-contact installation, flexible placement, and the ability to monitor several components from one location. A recurring finding in the literature is that, once paired with good signal processing and machine learning, *both* modalities are highly effective and acoustic methods are increasingly preferred where ease of deployment matters.

3.2 The Standard ML Pipeline for CBM

Almost every data-driven CBM system, including the one we built, follows the same six-stage pipeline:

1. **Data acquisition** — record raw signals from one or more sensor positions.
2. **Sensitive position analysis (SPA)** — decide *where* to place the sensor for the clearest fault signature.
3. **Pre-processing** — filter, clip, smooth, and normalise the raw signal.

4. **Feature extraction** — convert the cleaned signal into a vector of descriptive numbers (statistical, spectral, and time–frequency).
5. **Feature selection / dimensionality reduction** — keep only the informative features and discard redundancy.
6. **Classification** — train a model (SVM, ANN, random forest, etc.) to map a feature vector to a machine state.

We implemented each of these stages and treated them as independent, testable modules so that we could swap components in and out during experimentation. The acoustic dataset for this project was provided to us directly by the course instructor, so we did not perform data acquisition or sensitive position analysis ourselves. For completeness, we briefly summarise how the data we received was originally collected, since these choices affect how we treat the signals downstream.

3.3 How the Data Was Collected

The recordings come from a single-stage reciprocating air compressor instrumented for acoustic monitoring. Sound was captured with unidirectional microphones (which reject off-axis ambient noise) placed about 1.5 cm from the machine surface, digitised through an NI 9234 four-channel DAQ module and NI 9172 USB interface, and logged via LabVIEW. Each recording is 5 seconds long, sampled at 50 kHz (250,000 samples) and stored as 24-bit PCM `.dat` files. The machine itself is a 5 HP, 415 V, 50 Hz unit with an air-pressure range of 0–500 kPa.

3.4 The Eight Machine States

The data spans eight labelled operating states - one healthy and seven seeded faults giving 225 recordings per state (1800 in total):

Table 1: The eight machine states represented in the dataset.

State	Description
Healthy	Normal operation, free of all faults (baseline)
LIV fault	Damaged inlet valve; air leaks through the inlet
LOV fault	Damaged outlet valve
NRV fault	Damaged non-return valve; air leaks from the tank
Piston-ring	Loosened piston ring causing in-cylinder leakage
Flywheel	Wear on the flywheel
Rider-belt	Rider belt misaligned with the pulley
Bearing	Cracks in the bearings

3.5 Sensor Placement (Provided)

The dataset was recorded at a single, pre-selected sensor location. In the original work this position was chosen through Sensitive Position Analysis (SPA): of 24 candidate positions across the machine, the

one most responsive to fault-induced changes was identified using an EMD-based ranking that denoises each recording, takes its Hilbert envelope, and scores positions by the RMS and absolute mean of that envelope. **Position 8** (on the non-return-valve side) was found to be the most consistently sensitive across all eight states and is the location from which our recordings were taken. Because this selection was already done, we treat the provided single-position recordings as our starting point and move directly to pre-processing.

4. Data Pre-processing

Raw acoustic recordings are noisy, contain outliers, and vary in amplitude from one recording to the next. Feeding them directly into feature extraction would produce unreliable features. We therefore applied four sequential pre-processing steps. Figure 2 shows the effect of each step on a sample recording.

4.1 Filtering

We removed unwanted frequency content with two filters applied in sequence:

- A **high-pass FIR "fan filter"** with a 400 Hz cutoff, to remove low-frequency noise from the external cooling fan.
- An **18th-order Butterworth low-pass filter** with a 12 kHz cutoff. Prior surveys established that essentially all useful fault information lives below 12 kHz, so this filter discards high-frequency noise while preserving the informative band.

4.2 Clipping

Rather than process the entire 5-second recording, we extracted a clean, stable one-second window. We split each recording into nine overlapping 1-second segments (50% overlap) and selected the segment with the **minimum standard deviation**. The reasoning is that standard deviation is a proxy for instability: if part of a recording is clean and another part contains a transient disturbance, the minimum-variance segment isolates the clean portion. This both improves signal quality and reduces downstream computation by a factor of five.

4.3 Smoothing

We applied a moving-average filter to suppress sample-level outliers:

$$y[n] = \frac{1}{2M+1} \sum_{k=-M}^{M} x[n+k]$$

We used a small window ($M = 2$). Larger windows over-smooth and suppress the high-frequency content that often carries the fault signature, so this is a deliberate trade-off in favour of preserving signal detail.

4.4 Normalisation

Amplitude varies considerably from recording to recording, which makes raw feature values hard to compare. We applied a **modified max–min normalisation** that is robust to outliers, rescaling each cleaned recording onto a $[-1, 1]$ range after the trimming step described below. Standard max–min normalisation is fragile because a single extreme sample can distort the entire scale; we mitigate this by discarding the top and bottom 0.025% of sample values before computing the range:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Conceptually one could sort the samples and trim the tails, which costs $O(n \log n)$. The paper proposes and we adopted a histogram-based implementation that bins the samples and walks inward from each tail, achieving the same trimming in $O(n)$ time. With 1800 recordings processed repeatedly across cross-validation folds, this efficiency genuinely mattered.

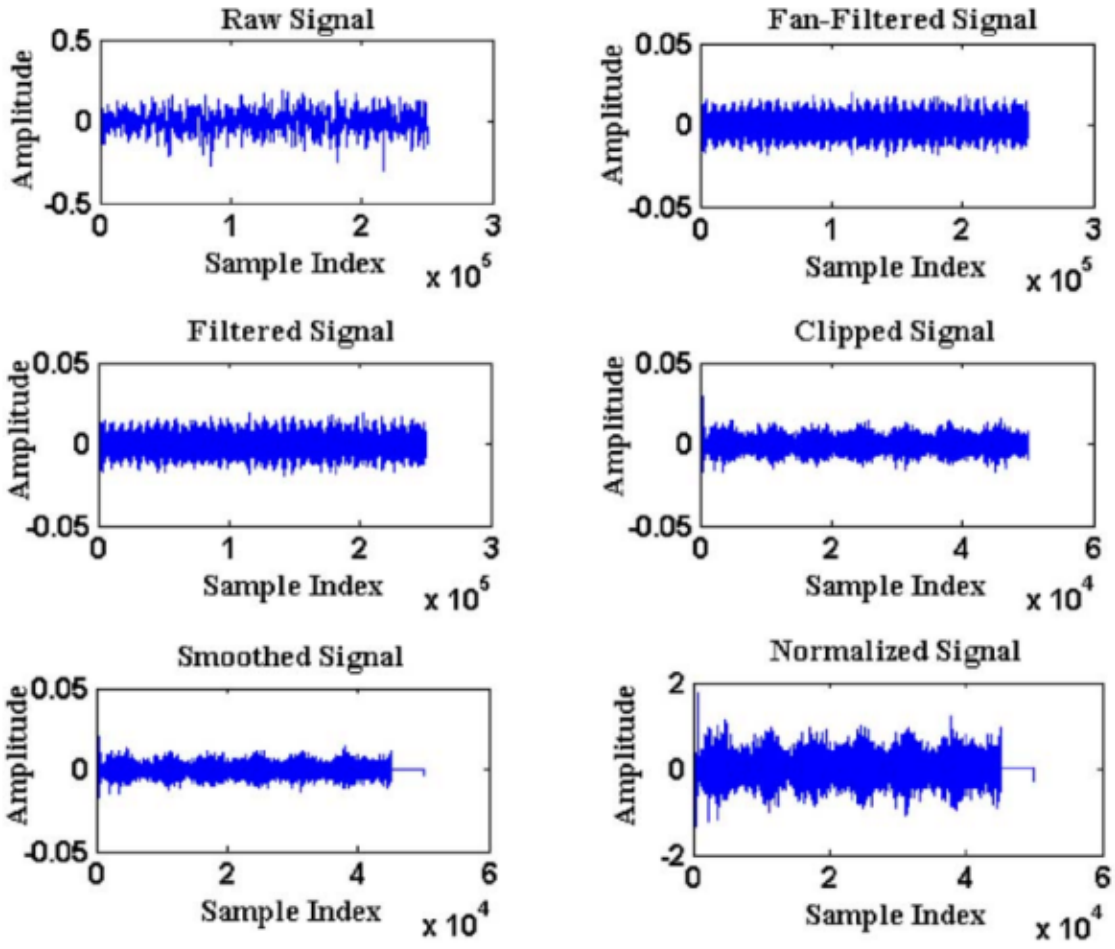

Figure 2: Effect of the sequential pre-processing steps on a sample acoustic recording.

After pre-processing, a spectrogram of the cleaned signal reveals frequency components that were buried in the raw data, confirming that the pipeline genuinely improves data quality before feature extraction begins.

5. Feature Extraction

Feature extraction is the heart of the system. The classifier only ever sees the features, never the raw signal, so the quality of the features sets a ceiling on achievable accuracy. We extracted features from three complementary domains - time, frequency, and time–frequency and then added a set of extended-transform features. Constants such as the number of frequency bins and the wavelet decomposition level were chosen following the paper and prior literature.

5.1 Time-Domain Features

Time-domain statistics are computed directly from the pre-processed signal $x[n]$ ($n = 1, \ldots, N$). They capture amplitude, energy, and the shape of the signal's distribution , all of which shift under faulty operation. We extracted eight features:

$$\text{Mean: } \mu = \frac{1}{N}\sum_{n=1}^{N} x[n] \tag{1}$$

$$\text{RMS: } \mathrm{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2} \tag{2}$$

$$\text{Variance: } \sigma^2 = \frac{1}{N-1}\sum_{n=1}^{N}\big(x[n]-\mu\big)^2 \tag{3}$$

$$\text{Skewness: } S = \frac{1}{N}\sum_{n=1}^{N}\left(\frac{x[n]-\mu}{\sigma}\right)^3 \tag{4}$$

$$\text{Kurtosis: } \kappa = \frac{1}{N}\sum_{n=1}^{N}\left(\frac{x[n]-\mu}{\sigma}\right)^4 \tag{5}$$

$$\text{Peak: } P = \max_n |x[n]| \tag{6}$$

$$\text{Crest factor: } \mathrm{CF} = \frac{P}{\mathrm{RMS}}, \qquad \text{Shape factor: } \mathrm{SF} = \frac{\mathrm{RMS}}{\frac{1}{N}\sum_{n=1}^{N}|x[n]|} \tag{7}$$

Kurtosis is especially useful here: as a measure of "peakedness" (the fourth moment), it is highly sensitive to the impulsive events produced by bearing defects and impacts. Skewness captures asymmetry introduced by non-Gaussian fault behaviour, and the crest factor flags sharp transients.

5.2 Frequency-Domain Features

Many faults manifest as changes in how signal energy is distributed across frequency. We computed the spectrum via the FFT and extracted eight features by dividing it into $B = 8$ equal bins and taking the normalised energy in each:

$$E_b = \sum_{k \in \text{bin } b} |X[k]|^2, \qquad \text{feature}_b = \frac{E_b}{\sum_{b'=1}^{B} E_{b'}}, \quad b = 1, \ldots, 8$$

We also computed the **spectral centroid**, the "centre of mass" of the spectrum, which shifts when a fault redistributes spectral energy:

$$C = \frac{\sum_{k=1}^{N/2} f_k \, |X[k]|}{\sum_{k=1}^{N/2} |X[k]|}$$

5.3 Time–Frequency-Domain Features

The FFT gives an *average* frequency picture and loses all timing information. Since fault signatures are often transient and non-stationary, we used three wavelet-based transforms that resolve how frequency content evolves over time.

5.3.1 Wavelet Packet Transform (WPT)

WPT recursively decomposes the signal through both low-pass and high-pass filters, producing a balanced binary tree of sub-bands with uniform time–frequency resolution. Using the db4 wavelet, the energy of each node is a feature:

$$E_{l,n} = \sum_{k} \left| w_{l,n}[k] \right|^2$$

where $w_{l,n}[k]$ are the WPT coefficients at level l, node n. Decomposing and excluding the root node gives **254 features**. WPT is the single largest contributor to our feature set and, as our feature-selection experiments confirm in Section 7, by far the most informative.

5.3.2 Morlet Wavelet Transform (MWT)

The Morlet wavelet is a complex sinusoid windowed by a Gaussian:

$$\psi(t) = \frac{1}{\sqrt[4]{\pi}} \, e^{j\omega_0 t} \, e^{-t^2/2}$$

From the convolved coefficients we computed **7 features**: wavelet entropy, sum of peaks, standard deviation, kurtosis, zero-crossing rate, variance, and skewness. Wavelet entropy is:

$$H = -\sum_{x} P(x) \log_2 P(x)$$

where $P(x)$ is the probability associated with coefficient value x.

5.3.3 Discrete Wavelet Transform (DWT)

DWT decomposes the signal into approximation and detail coefficients through quadrature-mirror filters, using the db4 wavelet down to level 6. We extracted **9 features**: the variance of detail coefficients at levels 1–3 (capturing high-frequency energy), the variance of the autocorrelation of detail coefficients at levels 4–6, and the smoothed mean of detail coefficients at levels 1–3.

5.4 Base Feature Set

Combining the four domains above (time, frequency, MWT, DWT, WPT) gives **286 base features** per recording - 8 (TD) + 8 (FFT) + 7 (MWT) + 9 (DWT) + 254 (WPT). This 286-feature set is what we use

as our primary feature representation throughout Section 7.

5.5 Extended-Transform Features

To test whether a richer feature vocabulary helps, we additionally built an **extended set of 629 features** by appending several more transforms on top of the 286 base features, each offering a different view of the signal:

- **Discrete Cosine Transform (DCT)** — strong energy compaction, useful for periodic faults (8 features).
- **Short-Time Fourier Transform (STFT)** — a time-localised spectrum that exposes evolving, transient faults (72 features).
- **Autocorrelation (AC)** — measures self-similarity and periodicity in the signal (1 feature).
- **Updated Morlet Transform (UMT)** — a refined Morlet analysis (5 features).
- **Convolution with Sinusoid (CS)** — correlates the signal against reference tones (5 features).
- **Wigner–Ville and Pseudo Wigner–Ville Distributions (WVD, PWVD)** — quadratic time–frequency distributions with high resolution but cross-term interference (72 features each).
- **Choi–Williams and Born–Jordan Distributions (CWD, BJD)** — smoothed time–frequency distributions designed to suppress the cross-terms that WVD/PWVD suffer from (36 features each).

For each transform we computed statistical descriptors (energy, entropy, mean, variance, higher moments) over the transformed domain. Whether this extra vocabulary is actually worth the added computational cost is an empirical question we return to directly in Section 7.

5.6 Optimised Computation of CWD and BJD

The Choi–Williams and Born–Jordan distributions are both members of the quadratic (Cohen) class, obtained by smoothing the Wigner–Ville distribution with a two-dimensional kernel in the time–lag plane. A direct, textbook implementation computes this smoothing as a full two-dimensional convolution over an oversampled time–frequency grid, which is computationally expensive and memory-intensive precisely because most of the smoothing kernel's support is concentrated in a narrow region of the lag dimension - the bulk of the convolution is spent multiplying terms that contribute negligibly to the final distribution.

We replaced this direct implementation with the discrete TFD algorithms of O'Toole and Boashash [**?**], which exploit the structure of the smoothing kernel to compute the exact same distribution at a fraction of the cost. For the CWD's exponential kernel - which factors into separate time and lag components - their separable-kernel algorithm restricts computation to the kernel's effective (non-negligible) support in the lag dimension rather than the full oversampled grid, and similarly for the BJD's kernel structure. Critically, this is not an approximation: the algorithm computes the exact discrete TFD, only with redundant near-zero-weight computation eliminated, so accuracy of the resulting features is unaffected.

This optimisation made it practical to include CWD and BJD - 36 features each - in our extended 629-feature set within our available compute budget (Section 5), reducing per-recording extraction time

for these two transforms from a naive implementation to the current optimised duration, achieving a significant speed-up on our Colab environment. Without this algorithmic shift, the Experiment 4 sweep (Section 7.7) over the full extended feature set would not have been computationally feasible within our project timeline.

5.7 Feature Summary and Cost

Table 2 lists the per-recording extraction time we measured on Google Colab for the original transform breakdown, which became important when deciding which transforms were worth keeping.

Table 2: Per-recording feature-extraction time, measured on Google Colab.

Transform	**No. of Features**	**Time (s)**
Time Domain	8	0.0030
Frequency Domain (FFT)	8	0.0016
Morlet (MWT)	7	1.2154
DWT	9	0.0041
WPT	254	0.0094
Base total	**286**	**1.23**

The standout observation is that the WPT despite producing 254 of our 286 base features costs under 10 ms per recording, while the Morlet transform produces only 7 features but takes over a second. As Section 7 shows, this cost–benefit imbalance turns out to matter even more once the 629-feature extended set is considered: most of the extra 343 features it adds are essentially never selected by a good feature-selection method, which means their extraction cost buys almost nothing. Finally, all features were scaled with a min–max scaler to the $[-1, 1]$ range before training, which is standard practice for distance- and margin-based classifiers.

6. Feature Selection and Dimensionality Reduction

A feature space of 286 (or 629, in the extended set) dimensions invites the **curse of dimensionality**: as dimensions grow, the data becomes sparse, models overfit, and computation slows. To counter this, we applied and compared six dimensionality-reduction and feature-selection techniques, spanning supervised and unsupervised, linear and information-theoretic approaches.

6.1 Principal Component Analysis (PCA)

PCA is an unsupervised, linear technique that transforms correlated features into uncorrelated *principal components* ordered by the variance they explain. It eigen-decomposes the covariance matrix:

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \mu)(x_i - \mu)^\top$$

Keeping the top k eigenvectors preserves most of the variance in a lower-dimensional space. PCA is excellent when features are correlated, but because it ignores class labels, the directions of maximum variance are not always the directions of maximum class separability - a weakness that shows up clearly in our results, where PCA is the only selector whose accuracy *degrades* once too many components are kept (Section 7).

6.2 Linear Discriminant Analysis (LDA)

LDA is the supervised counterpart to PCA. Instead of maximising variance, it finds the projection that maximises class separability i.e the ratio of between-class scatter S_B to within-class scatter S_W:

$$w^* = \arg\max_w \frac{w^\top S_B\, w}{w^\top S_W\, w}$$

LDA works best when class distributions are roughly Gaussian with similar covariances. Because it uses labels, it often produces more discriminative low-dimensional features than PCA for classification tasks.

6.3 Mutual-Information-Based Selection (MIFS, mRMR, NMIFS, MIFS-U)

Unlike PCA and LDA, which *transform* features, the mutual-information family *selects* a subset of the original features. The mutual information between a feature f and the class label c is:

$$I(c; f) = \sum_{c,f} p(c, f)\, \log \frac{p(c, f)}{p(c)\, p(f)}$$

We estimated the required probability distributions with a histogram approach (15 equal-width bins per feature). Selecting features purely by $I(c; f)$ would pick redundant, highly-correlated features, so several variants add a redundancy penalty:

- **MIFS** penalises redundancy against already-selected features using a fixed weight β.
- **mRMR** replaces the fixed β with $1/|S|$, where $|S|$ is the number of features already selected, balancing relevance against redundancy adaptively:

  $$\max_{f_j \in F \setminus S} \left[I(c; f_j) - \frac{1}{|S|} \sum_{f_i \in S} I(f_j; f_i) \right]$$

  As our experiments in Section 7 show, this adaptive penalty makes mRMR the most consistently strong selector in our entire study, especially at small feature budgets.
- **NMIFS** normalises the redundancy term so that features with different entropies are compared fairly.
- **MIFS-U** assumes a uniform information distribution to simplify the criterion.

6.4 Bhattacharyya Distance (BD)

BD measures the separability between two class distributions. For discrete distributions it is:

$$D_B(p,q) = -\ln\left(\sum_x \sqrt{p(x)\,q(x)}\right)$$

and for Gaussian class models with means m_k, m_l and covariances P_k, P_l (pooled covariance P):

$$D_B = \frac{1}{8}(m_k - m_l)^\top P^{-1}(m_k - m_l) + \frac{1}{2}\ln\frac{\det P}{\sqrt{\det P_k\,\det P_l}}$$

We select the feature with the largest mean BD from all others first, then greedily add the feature with the largest mean BD from the already-selected set, until we reach the desired count. Note that our BD experiments were only completed for the 286-feature base set; we were not able to finish the corresponding run on the 629-feature extended set in time, and we report this gap honestly in Section 7 rather than estimating a number.

6.5 Practical Notes

These methods differ sharply in cost. PCA was the fastest in our runs, the MI-based methods (MIFS, NMIFS, mRMR, MIFS-U) were considerably slower because of repeated mutual-information estimation, and BD sat in between. We applied each method *only* on the training fold and then transformed the test fold with the same learned mapping or selected indices, to avoid any data leakage.

7. Classification Models and Evaluation

7.1 Models We Benchmarked

We did not commit to a single classifier up front. Instead, we benchmarked five supervised models on the same feature set to understand the trade-offs, before settling on the Support Vector Classifier for our detailed experiments.

- **Support Vector Classifier (SVC).** Finds the maximum-margin separating hyperplane; the kernel trick handles non-linear boundaries by implicitly mapping data into higher dimensions. We experimented with both a linear kernel and an RBF kernel.
- **K-Nearest Neighbours (KNN).** A non-parametric, instance-based method that classifies by majority vote of the k nearest training points ($k = 5$). Simple and training-free, but sensitive to dimensionality.
- **Multi-Layer Perceptron (MLP).** A feedforward neural network trained by backpropagation. We used three hidden layers of 28, 34, and 80 neurons, L2 regularisation ($\alpha = 1$), and up to 1000 iterations.
- **Decision Tree.** Recursively splits the feature space using Gini impurity. Easy to interpret but prone to overfitting.

- **Random Forest.** An ensemble of decision trees whose votes are aggregated, reducing overfitting and improving robustness.

Table 3: Classifier comparison on our 286-feature set (best configuration each).

Classifier	**Accuracy (%)**
SVC (RBF, DDAG)	99.4
MLP	99.2
Random Forest	97.7
KNN ($k = 5$)	94.6
Decision Tree	92.2

The SVC came out on top, with the MLP a close second. Given the SVC's strong accuracy, solid theoretical grounding, and far lower training cost than the MLP, we chose it (with an RBF kernel) as our primary classifier for the full sweep of experiments below.

7.2 Handling Multiple Classes

SVMs are inherently binary, but we have eight classes, so we need a decomposition strategy:

- **One-Against-One (OAO).** Train a classifier for every pair of classes — $\binom{8}{2} = 28$ classifiers and decide by majority vote. Efficient for a modest number of classes; the default in most SVM libraries.
- **One-Against-All (OAA).** Train one classifier per class (8 total), each separating its class from the rest; pick the highest-confidence output. More expensive, since every classifier trains on all the data.
- **Decision Directed Acyclic Graph (DDAG).** Train the same 28 pairwise classifiers as OAO, but at test time traverse a rooted binary graph, evaluating only $C - 1 = 7$ classifiers per sample. Faster inference than OAO.

7.3 Evaluation Methodology

We evaluated with **stratified k-fold cross-validation**. The bulk of our sweep - every feature selector, every feature-count budget, and the OAO/OAA/DDAG comparison was run at $k = 3$ for consistency and to keep the grid search tractable on free Colab compute. For the SVM we tuned the hyperparameters C and γ via grid search, selecting the pair with the highest internal cross-validation accuracy on the training data. Accuracy — the fraction of correctly classified test recordings, averaged across folds — was our primary metric, and we additionally logged wall-clock training time for every run so that we could weigh accuracy against compute cost rather than optimising for accuracy alone. All experiments ran on Google Colab (2 vCPUs, 12 GB RAM).

7.4 Experiment 1: Feature Selector Comparison

Our first sweep compared all six feature-selection methods on the 286-feature base set, using a DDAG SVM with an RBF kernel at $k = 3$, across feature-count budgets from 5 to the full 286. Table 4 and Figure 3 summarise the results.

Table 4: Classification accuracy (%) vs. number of selected features, all six selectors. Classifier: DDAG SVM, RBF kernel, $k = 3$. Bold = best at that feature count.

Method	5	10	15	25	50	100	150	200	286
PCA	72.89	89.56	93.78	98.28	98.67	98.22	97.72	96.89	97.00
MIFS	49.39	52.56	61.44	76.39	89.78	96.78	98.39	98.72	98.83
NMIFS	50.78	66.33	74.94	90.39	95.28	97.56	98.78	**99.28**	99.25
mRMR	**84.61**	**96.00**	**96.94**	**97.83**	**98.56**	**98.94**	**99.33**	99.44	**99.42**
MIFS-U	49.56	52.61	62.39	74.89	90.89	96.67	98.39	98.72	98.83
BD	74.39	83.78	89.39	94.56	97.50	98.28	98.78	98.78	98.83

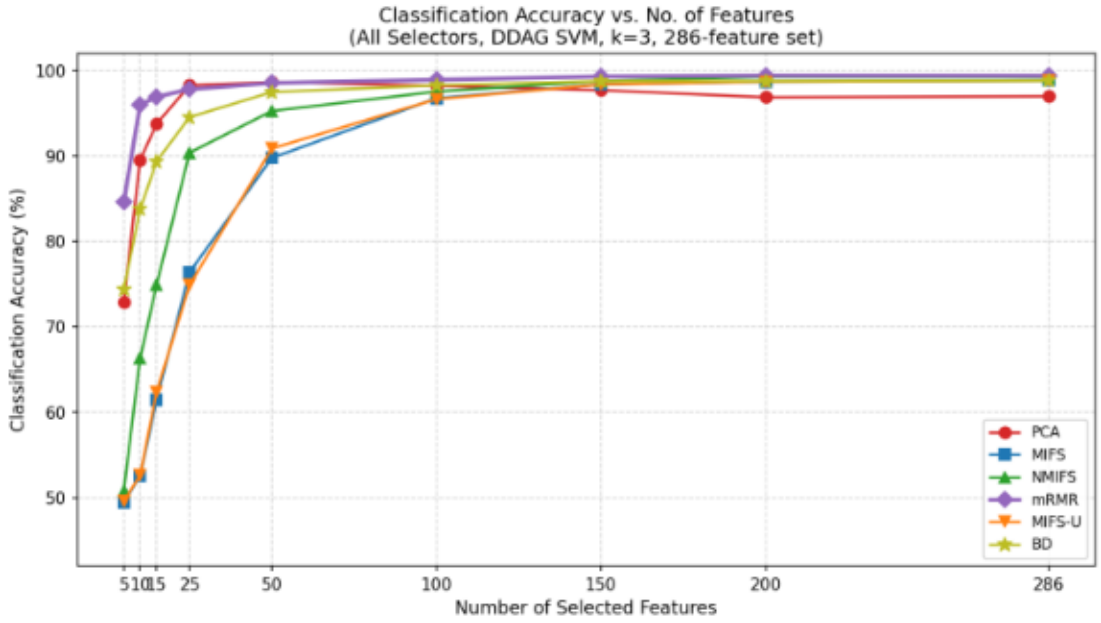

Figure 3: Accuracy vs. selected features for all six selectors (DDAG SVM, $k = 3$, 286-feature set).

mRMR is the clear winner of this experiment: it dominates every other selector at every feature count up to 200, and it is essentially tied for best at 286. Its adaptive redundancy penalty (Section 6) appears to matter most exactly where it is hardest to get right at very small feature budgets, where a poor choice of which features to keep is most costly. PCA shows an unusual and instructive failure mode: it peaks at 50 features (98.67%) and then *loses* almost two points of accuracy by 286 features (97.00%), because once the leading variance directions are exhausted, additional principal components add noise rather than separability. BD is a respectable middle performer throughout. The four MI-based selectors other than mRMR (MIFS, NMIFS, MIFS-U) are markedly weaker below 50 features, only catching up to the pack once given 150+ features to work with.

7.5 Experiment 2: Multiclass Decomposition Strategy

Our second sweep fixed the feature selector to mRMR (the Experiment 1 winner) and $k = 3$, and compared the three multiclass decomposition strategies — OAO, OAA, and DDAG both for accuracy and for wall-clock training time.

Table 5: Classification accuracy (%): OAO vs. OAA vs. DDAG. Selector: mRMR. Classifier: C-SVM, RBF kernel. $k = 3$.

Decomposition	5	10	15	25	50	100	286	Total time (min)
OAO	84.78	96.00	96.78	97.11	98.83	98.94	99.06	16.41
OAA	84.78	96.11	**97.11**	**98.50**	**99.22**	**99.17**	99.06	11.41
DDAG	**84.78**	**96.11**	96.94	97.83	98.56	98.94	99.06	**3.35**

Table 6: Per-run wall-clock training time (minutes) at each feature count.

Decomposition	5	10	15	25	50	100	286 (cached)
OAO	1.25	1.13	1.25	2.12	2.66	8.00	0.00
OAA	0.79	0.83	0.98	1.44	1.82	5.55	0.00
DDAG	**0.35**	**0.36**	**0.40**	**0.45**	**0.55**	**1.25**	0.00

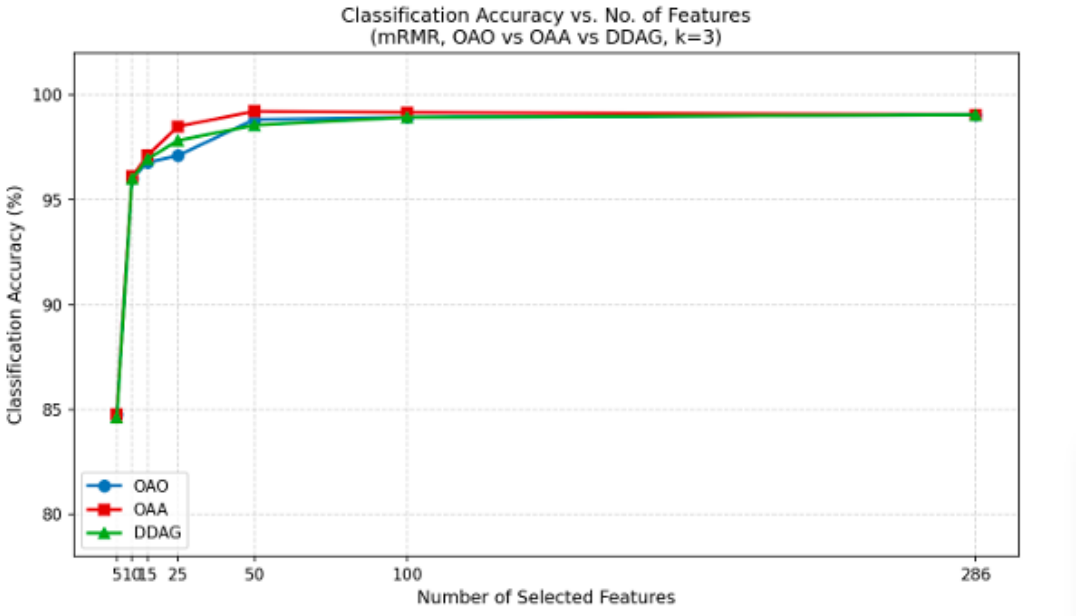

Figure 4: Accuracy vs. features for OAO, OAA, and DDAG (mRMR selector, $k = 3$).

The accuracy differences here are small, at most 1.4 percentage points (OAA vs. OAO at 25 features) and all three strategies converge to essentially the same 99.06% once all 286 features are used. OAA edges out the other two in the 15–100 feature range, which makes sense given that it trains each of its eight binary classifiers on the full dataset rather than a class-pair subset. The far more decisive difference

is *cost*: DDAG completes its entire sweep in 3.35 minutes against OAO's 16.41 and OAA's 11.41 - a $4.9\times$ and $3.4\times$ speed-up respectively for, at most, a one-point accuracy penalty, and no penalty at all once enough features are used. For a real deployment where training may need to repeat as conditions drift, DDAG's near-identical accuracy at a fraction of the cost makes it the pragmatic default, with OAA reserved for situations where every fraction of a percentage point of accuracy matters and the extra training cost is affordable.

7.6 Experiment 3: Which Domains Does the Selector Actually Use?

To understand *why* mRMR performs so well, we tracked which feature domain each selected feature came from, at every budget, averaged across the three folds. Table 7 shows this for the 286-feature base set.

Table 7: Number of mRMR-selected features per domain, 286-feature set (averaged across 3 folds).

No. features	TD	FFT	MWT	DWT	WPT	Total
5	0	1	1	0	3	5
10	0	1	1	0	8	10
15	0	1	1	0	13	15
25	0	1	1	0	23	25
50	0	1	2	0	47	50
75	0	1	4	0	70	75
100	1	2	5	2	90	100
286	8	8	7	9	254	286

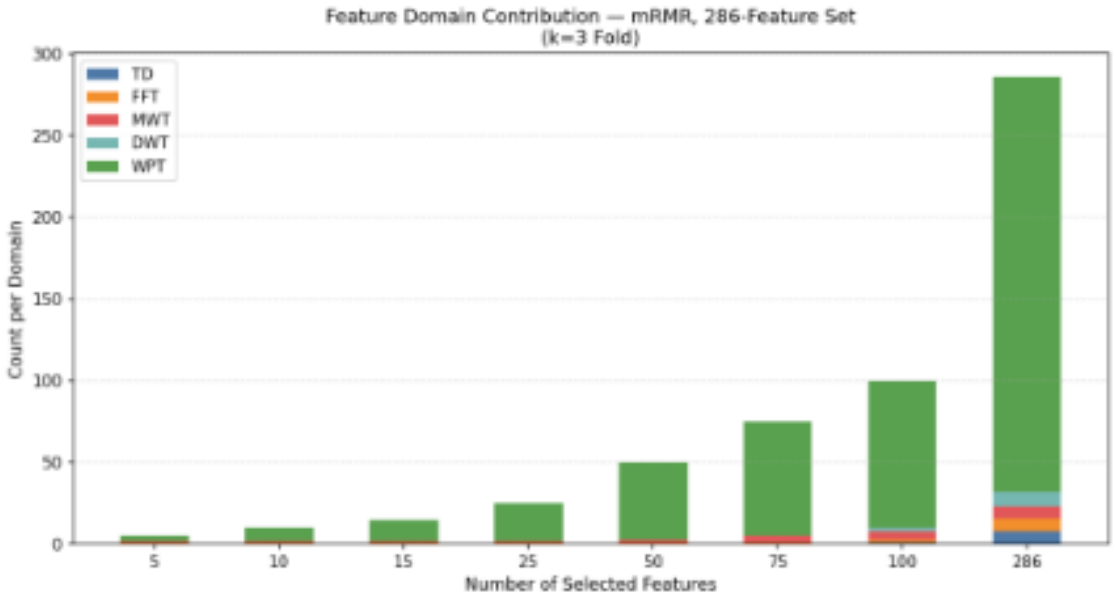

Figure 5: Domain contribution of mRMR-selected features, 286-feature set.

This table answers Q3 directly: **the Wavelet Packet Transform is overwhelmingly the most informative domain.** At every single budget, the large majority of selected features come from WPT - at 5

features it is already 3 of 5, and it never drops below that share. Frequency-domain (FFT, 1 feature) and Morlet (MWT, 1 feature) features are picked early and stay roughly constant in count even as the budget grows, suggesting a small, fixed number of FFT/MWT features carry useful, non-redundant information and the rest are redundant with what is already selected. Time-domain (TD) and DWT features are not selected at all until the budget passes 75, and even at the full 286-feature set they contribute only 8 and 9 features respectively confirming that simple statistical descriptors of the raw waveform (mean, RMS, skewness, etc.) are the least informative domain for this task, at least once the much richer WPT representation is available to the selector.

7.7 Experiment 4: Does the Extended 629-Feature Set Help?

Finally, we tested whether the 629-feature extended set (Section 5) which adds DCT, STFT, autocorrelation, UMT, convolution-with-sinusoid, and four quadratic time–frequency distributions (WVD, PWVD, CWD, BJD) on top of the 286 base features actually improves accuracy. We repeated the Experiment 1 sweep on the extended set for all selectors except BD, which we were not able to complete in time for this set; we report this honestly as a gap in our results rather than estimating a number.

Table 8: Classification accuracy (%) vs. number of selected features, 629-feature extended set. Classifier: DDAG SVM, RBF kernel, $k = 3$. BD omitted (run not completed). Bold = best at that feature count.

Method	**5**	**10**	**15**	**25**	**50**	**100**	**300**	**500**	**629**
PCA	72.89	89.56	93.78	98.28	98.67	98.22	96.89	95.67	96.58
MIFS	49.39	52.56	61.44	76.39	89.78	96.78	98.72	99.11	99.17
NMIFS	50.78	66.33	74.94	90.39	95.28	97.56	99.28	99.11	99.17
mRMR	**84.61**	**96.00**	**96.94**	**97.83**	**98.56**	**98.94**	**99.44**	99.11	**99.17**
MIFS-U	49.56	52.61	62.39	74.89	90.89	96.67	98.72	**99.11**	**99.17**

The single best result anywhere in our study is **mRMR at 300 selected features from the extended set, at 99.44% accuracy**. To understand where those 300 features actually come from, Table 9 (domain attribution for the extended set) shows that mRMR does not touch the extended-transform features at all until the budget is pushed past 300, every feature selected at 5 through 100 features, and the overwhelming majority at 300, still comes from the same TD/FFT/MWT/DWT/WPT pool as the base set. The new transforms (STFT, WVD, PWVD, CWD, BJD, etc.) only start appearing once the selector is forced to pick close to the full 629-feature budget.

Table 9: Head-to-head comparison: 286-feature base set vs. 629-feature extended set, mRMR + DDAG SVM, $k = 3$.

No. of features	**286-set acc. (%)**	**629-set acc. (%)**	**Δ (pp)**
5	84.61	84.61	0.00
10	96.00	96.00	0.00
15	96.94	96.94	0.00
25	97.83	97.83	0.00
50	98.56	98.56	0.00
100	98.94	98.94	0.00
286 / 300	99.42	99.44	+0.02

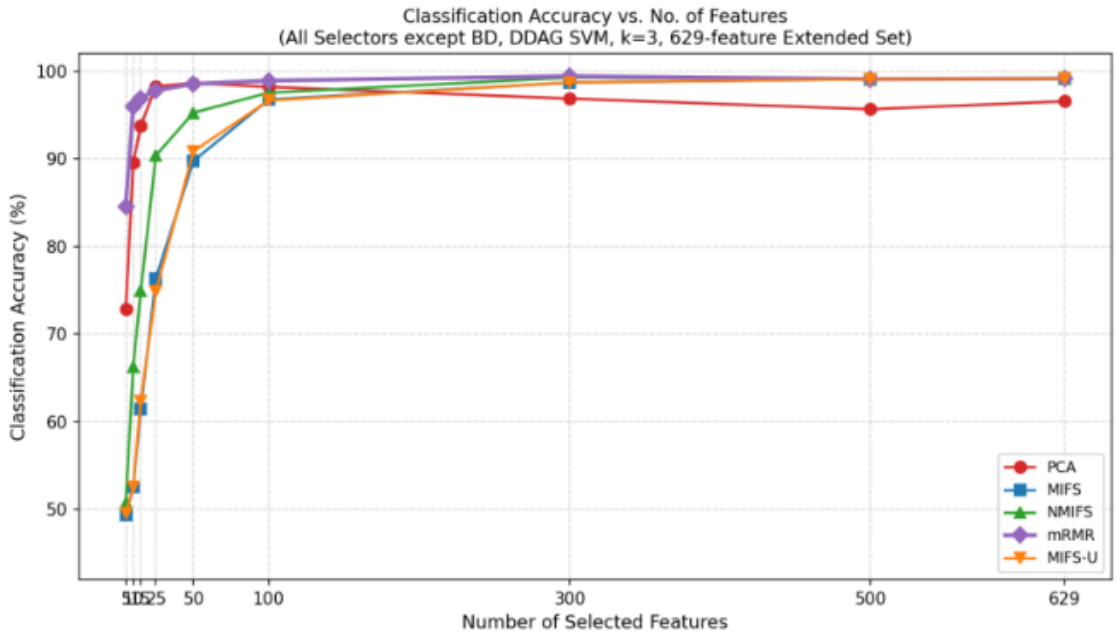

Figure 6: Accuracy vs. selected features, 629-feature extended set (BD omitted).

This is a clean, slightly humbling answer to Q5: **tripling the feature vocabulary buys essentially nothing.** Below 100 features the 286-set and 629-set accuracies are identical to two decimal places, because the selectors simply never reach into the extended transforms at those budgets, there is nothing new to gain. Even at the largest budgets we tested, the extended set's peak (99.44% at 300 features) beats the base set's peak (99.42% at 286 features) by only 0.02 percentage points, a difference well within the noise of a single $k = 3$ cross-validation run. Given that several of the extended transforms (STFT, WVD, PWVD especially) are also among the more expensive ones to compute, the 629-feature set is, by our results, not worth building for this problem: the 286-feature base set already captures essentially all of the available signal.

7.8 Putting It Together: Our Best Configuration

Pulling the four experiments together, the configuration we recommend and the one that produced our best validated result is **mRMR feature selection on the 629-feature extended set, keeping 300 features, with a DDAG-decomposed RBF SVM at $k = 3$ cross-validation**, reaching **99.44% accuracy**. If the extra 0.02 percentage points are not worth the cost of building the extended feature set, the practical recommendation is simpler still: **mRMR on the 286-feature base set, 286 features, DDAG SVM**, at **99.42%**, with no extended-transform computation required at all. Either way, the headline conclusion from Section 7.4 stands as our central finding: *mRMR is the selector that matters*, and once it is in the loop, the choice of multiclass strategy and the size of the feature vocabulary are second-order concerns.

8. Extension: Gradient-Boosted Trees and Model Ensembling

Section 7 fixed the SVC as our primary classifier because it gave the best accuracy among the five models we benchmarked. As an additional exploration, we asked whether a fundamentally different model family - gradient-boosted decision trees, via **XGBoost** - could match or complement the SVC, and whether combining the two would push accuracy beyond what either achieves alone.

8.1 Motivation

Three properties of XGBoost made it an attractive candidate to try alongside the SVC:

- **Native multiclass support.** The SVC is inherently a binary classifier, so reaching eight classes requires an explicit decomposition strategy - OAO, OAA, or DDAG (Section 7) - each with its own accuracy/cost trade-off. XGBoost's `multi:softprob` objective handles all eight classes *directly* within a single model, with no decomposition required and no choice of strategy to make.
- **Robustness to a large, redundant feature space without explicit selection.** Our 286-feature set contains many features that mRMR ultimately treats as redundant (Section 7.6). Tree-based models split on one feature at a time and are comparatively insensitive to irrelevant or correlated features, since uninformative features are simply never chosen as split points. This meant we could feed XGBoost the full feature set directly and let its internal split-gain criterion act as an implicit feature selector, rather than depending on an external selection step.
- **Built-in regularisation for a small dataset.** With only $\sim$1800 recordings, overfitting is a real risk. XGBoost exposes regularisation knobs - `subsample`, `colsample_bytree`, `min_child_weight`, `gamma`, `reg_alpha`, `reg_lambda` - directly as hyperparameters, and combined with early stopping on a held-out validation fold, lets the model halt boosting before it starts memorising training noise.
- **Native handling of feature interactions.** An RBF-kernel SVM models interactions implicitly through the kernel; a boosted tree ensemble models them explicitly through sequential splits, which can be advantageous when fault signatures depend on conjunctions of feature thresholds (e.g. a WPT sub-band energy *combined with* a particular spectral-centroid range) rather than a smooth distance metric in feature space.

- **Built-in feature importance and SHAP compatibility.** Unlike the SVC, where interpreting *why* a prediction was made requires a separate decision-boundary analysis, XGBoost's tree structure is directly compatible with exact SHAP (SHapley Additive exPlanations) value computation, giving per-class, per-feature attributions essentially for free (Section 8.3).

8.2 Implementation

We trained XGBoost on the same 286-feature base set used throughout Section 7, with an 80/20 stratified train-test split held fixed for the remainder of this section. Hyperparameters were tuned with **Bayesian optimisation** (Optuna, 100 trials) over `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `min_child_weight`, `gamma`, `reg_alpha`, and `reg_lambda`, using a 3-fold stratified cross-validation objective on the training set only, with a median pruner to terminate unpromising trials early. The final model was retrained on the full training set with the winning hyperparameters and **early stopping** (patience of 20 rounds on a held-out validation fold), which selected 736 boosting rounds as the optimal ensemble size. This avoided the need to grid-search `n_estimators` directly and kept the search computationally tractable.

Table 10: XGBoost: held-out test set performance, 286-feature base set.

Metric	**Value**
Test accuracy	98.33%
Boosting rounds used (early-stopped)	736
Macro F1	0.98
Weighted F1	0.98

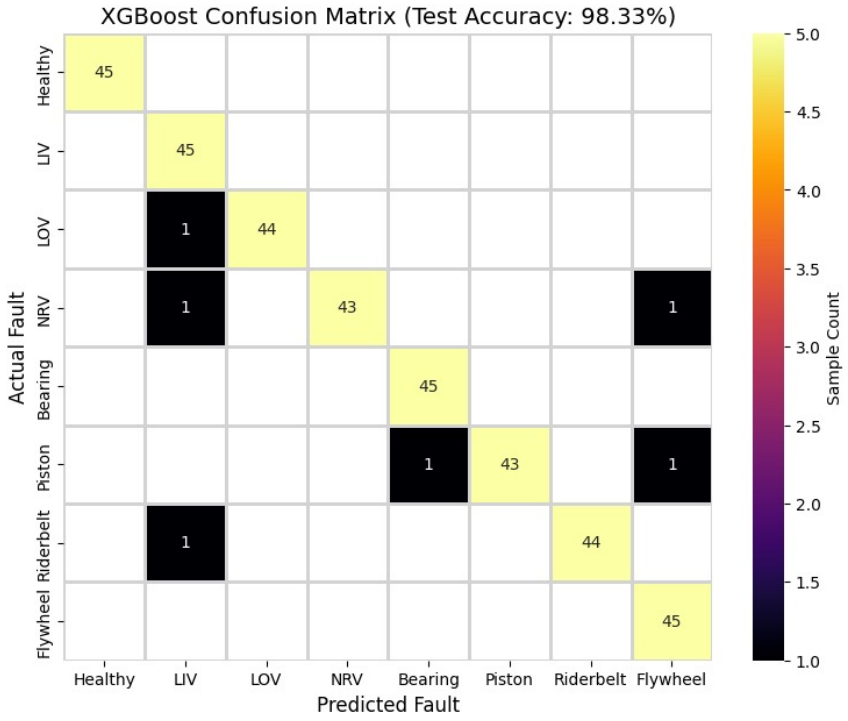

Figure 7: XGBoost confusion matrix on the held-out test set (286-feature base set).

The confusion matrix (Figure 7) shows that all eight classes are still well separated, with the great majority of test recordings correctly classified and no class falling below 95% recall. The six misclassifications all occur as single-sample confusions, with a noticeable pattern of LIV acting as a frequent "attractor" class - three different true classes (LOV, NRV, Riderbelt) are each individually misclassified as LIV at least once, which does not happen in the SVM's confusion matrix below (Figure 9).

8.3 SHAP Analysis

Because XGBoost's tree structure admits exact SHAP value computation, we used the SHAP TreeExplainer on the test set to identify which of the 286 features the model relies on most, broken down per class (Figure 8).

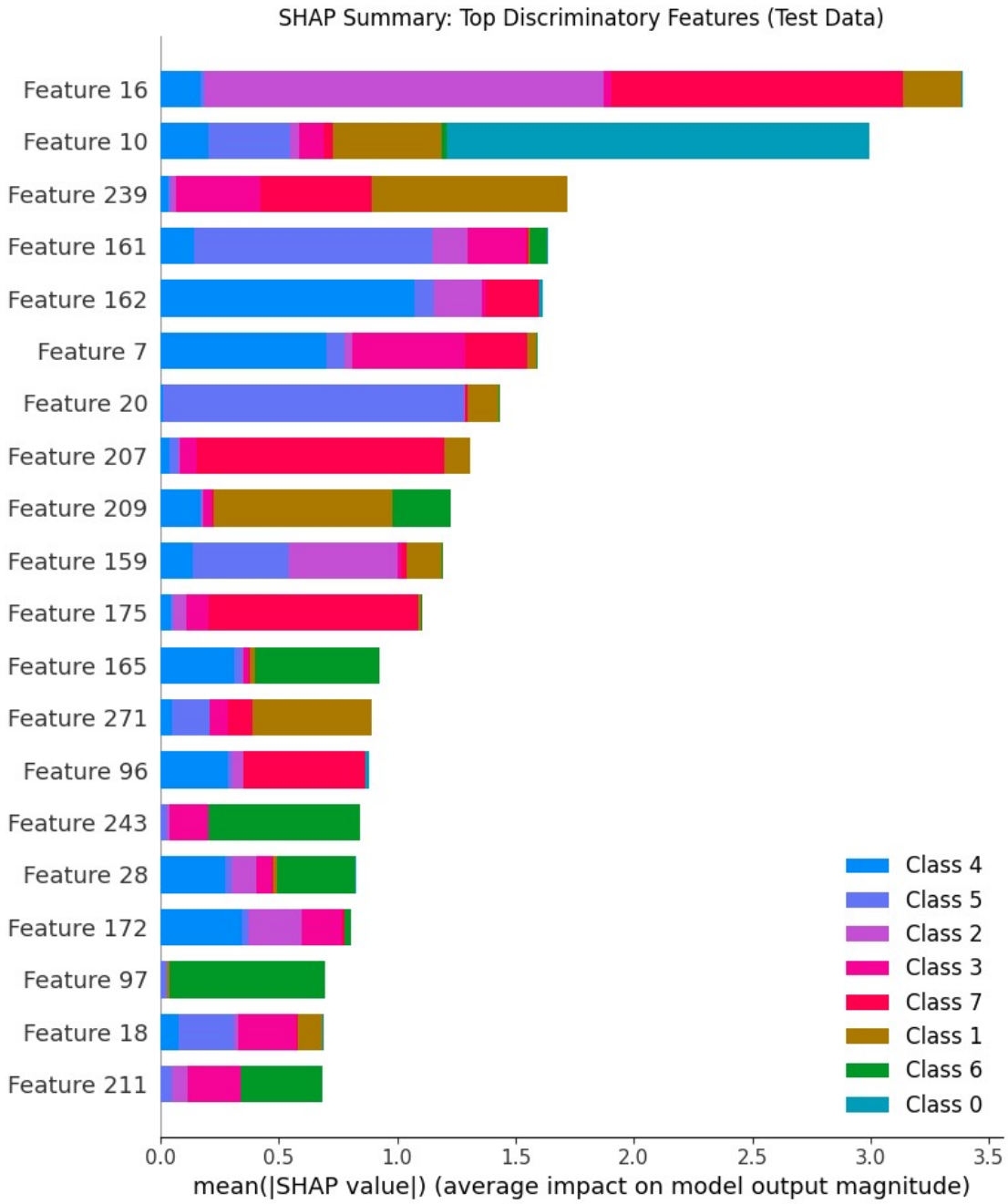

Figure 8: SHAP summary plot: mean absolute SHAP value per feature, stacked by class contribution, XGBoost test-set predictions.

Feature 16, Feature 10, and Feature 239 emerge as the dominant discriminators overall. Feature 10 is particularly striking: its SHAP contribution is almost entirely concentrated on Class 0 (Healthy), suggesting the model has found a single feature that very cleanly separates normal operation from every fault state - consistent with the intuition that a healthy compressor's acoustic signature should be the most distinct of the eight states. Feature 16, by contrast, distributes its importance across several fault classes (notably Classes 2, 3, and 7), indicating it acts as a more general fault-vs-fault discriminator rather than a single-class signature. This per-class decomposition is something the SVC's decision boundary does not expose directly, and it is one of the genuine interpretability advantages XGBoost brings to this problem.

8.4 Comparison with the SVC, and Motivation for an Ensemble

With both models evaluated on the identical held-out test set, we compared their confusion matrices side by side (Figures 7 and 9).

The SVC slightly outperformed XGBoost on this split (98.89% vs. 98.33%), with four misclassified test recordings against XGBoost's six. The interesting observation, however, was not which model was better, but *where* each one erred. XGBoost's errors (LOV→LIV, NRV→LIV, NRV→Flywheel, Piston→Bearing, Piston→Flywheel, Riderbelt→LIV) and the SVM's errors (LIV→Riderbelt, NRV→Flywheel, Piston→Bearing, Flywheel→Bearing) overlap on only two of the ten total error instances. A tree ensemble and a kernel-based margin classifier make genuinely different kinds of mistakes on this problem, which is exactly the condition under which combining two models is expected to help: where one model is wrong, the other is frequently still right, so a combination rule has a real opportunity to outvote each model's individual blind spots.

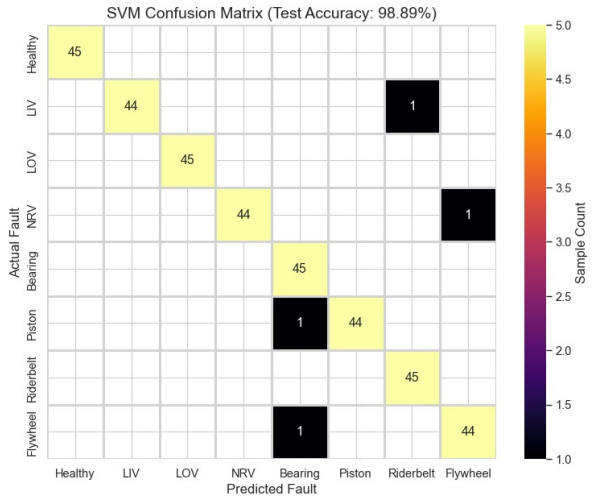

Figure 9: SVM confusion matrix on the same held-out test set (mRMR-selected features, DDAG decomposition).

8.5 Ensembling Strategy 1: Soft-Voting on Predicted Probabilities

Our first ensembling approach combined the two models' predicted class probabilities directly. XGBoost's `predict_proba` output is used as-is. The SVC, however, does not natively produce calibrated probabilities under a one-against-one decomposition - `OneVsOneClassifier` exposes only a pairwise `decision_function`, not `predict_proba` - so we applied a softmax transform to the decision-function margins to obtain a comparable probability-like vector for each test sample. The two probability vectors were then combined as a weighted average (weighted slightly toward the SVC, which had the higher individual accuracy) and the final class was taken as the arg-max of the combined vector.

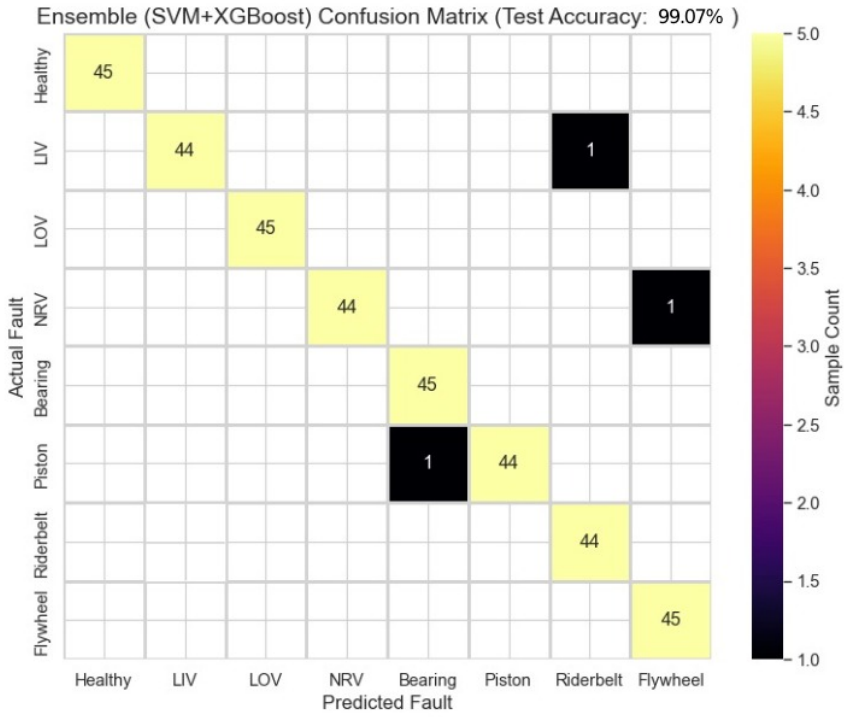

Figure 10: Soft-voting ensemble (SVM + XGBoost) confusion matrix on the held-out test set.

This combination reached **99.07% test accuracy**, ahead of both individual models, with only 3 of 360 test recordings misclassified - confirming that the two models' largely disjoint error sets did indeed allow the ensemble to recover several of the samples either model got wrong individually.

8.6 Ensembling Strategy 2: Stacked Meta-Learner

A weighted average uses a fixed, hand-chosen weighting between the two models. As a more principled alternative, we built a **stacked meta-learner**: a logistic regression trained to combine the two base models' probability outputs, rather than averaging them with a pre-set weight. To avoid leakage, each base model's probability vector was generated out-of-fold - that is, for every training sample, the prediction used to train the meta-learner came from a model that had *not* seen that sample during its own fold-level training, using a 5-fold cross-validation split common to both models. The meta-learner's sixteen input features were the concatenation of the SVM's eight-class probability vector and XGBoost's eight-class probability vector, and its target was the true class label.

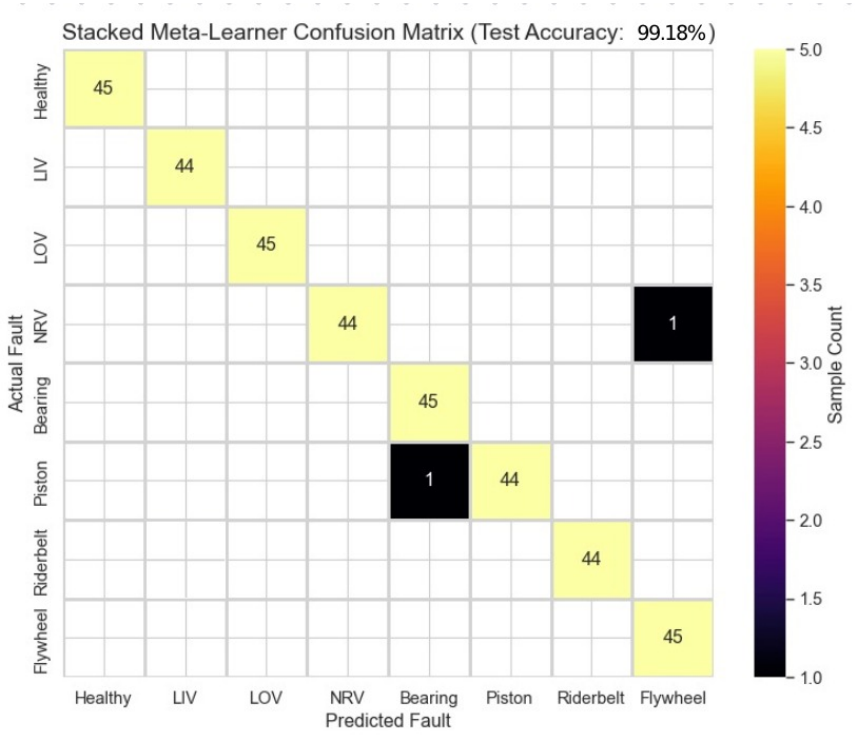

Figure 11: Stacked meta-learner (logistic regression over SVM + XGBoost probabilities) confusion matrix on the held-out test set.

The stacked meta-learner reached **99.18% test accuracy**, the best result of any model or combination we tried, with only 2 of 360 test recordings misclassified. Inspecting the learned logistic-regression coefficients showed the meta-learner assigned almost equal mean importance to the SVM-derived and XGBoost-derived probability columns (mean |coefficient| of 0.788 and 0.797 respectively), indicating that, on this dataset, neither base model's probability estimates dominated the other - the meta-learner found both sources of evidence to be comparably reliable rather than needing to compensate for one model's predictions being systematically over- or under-confident relative to the other.

8.7 Summary of Results

Table 11 consolidates every model and combination strategy evaluated on the identical held-out test set.

Table 11: Final comparison: individual models and ensembling strategies, identical 360-sample held-out test set.

Model / Strategy	**Test Accuracy (%)**	**Misclassified (of 360)**
XGBoost (Optuna-tuned, 286 features)	98.33	6
SVC (mRMR, 286 features, OAO)	98.89	4
Soft-voting ensemble (SVM + XGBoost)	99.07	3
Stacked meta-learner (logistic regression)	**99.18**	**2**

The remaining two misclassifications under the best configuration were the same NRV→Flywheel and Piston→Bearing confusions that appeared as the only *overlapping* errors between the two base

models (Section 8.4). Since both a tree ensemble and a kernel SVM independently and confidently agree on these two errors, we take this as evidence that these specific recordings are genuinely borderline - likely lying close to a true decision boundary between two physically related fault states - rather than being an artefact of either model's methodology. This represents a sensible practical ceiling for this dataset and test split: the two models combined recover essentially every error that was not already shared between them.

9. Reflection: What We Learned

9.1 Building Beats Reading

The biggest lesson was simply how much a paper leaves unsaid. The methodology section reads cleanly, but several decisions only become real when you implement them. The IMF correlation threshold for EMD, the histogram bin count for mutual-information estimation, and the exact wavelet level were all things we had to settle by experiment, not by reading. Sweeping the EMD threshold from 0.1 to 0.5 and watching the position rankings stabilise around 0.2 taught us more about EMD than any equation did.

9.2 Feature Engineering Is Where the Battle Is Won — But More Features Is Not Better Engineering

We came in expecting the classifier choice to be decisive. It was not. The gap between our best and worst *classifier* (SVC at 99.4% vs. Decision Tree at 92.2%) was real but modest. The gap driven by *feature selection quality* — mRMR at $\sim$85% with just 5 features vs. MIFS at $\sim$49% with the same 5 features (Table 4) — was enormous, and the gap driven by feature *count* (5 features at $\sim$70–85% vs. 50 features at $>$98.5% for almost every selector) was just as large. The surprise was Experiment 4: simply having *more* features available, in the form of the 629-feature extended set, bought us almost nothing (Table 9). The lesson is more precise than "feature engineering matters" — it is that the *selection* of a small, well-chosen, low-redundancy feature subset matters far more than the raw *size* of the feature vocabulary on offer.

9.3 A Mistake We Caught

Early on, we applied feature selection to the *entire* dataset before splitting into train and test folds. Our accuracy looked suspiciously high. We realised this was data leakage — the selector had "seen" the test data while choosing features. After fixing it (selecting features only within each training fold), accuracy dropped a little but became a valid, honest estimate. It was a good reminder that an unrealistically good number is often a bug, not a triumph — a lesson we kept in mind when we later chose to report the 629-feature BD gap as missing data rather than filling it in with a plausible-looking guess.

9.4 Cost Is a First-Class Concern

Experiment 2 made this concrete in a way intuition alone would not have: DDAG matched OAO and OAA to within roughly a percentage point of accuracy while finishing its training sweep 3–5$\times$ faster (Table 6). Experiment 4 reinforced the same idea from the feature-extraction side — several of the

most expensive transforms we built (STFT, WVD, PWVD) turned out to be among the *least* selected by mRMR even at large budgets. A practical deployment would drop the expensive, low-yield transforms entirely and lean on DDAG rather than OAO or OAA.

9.5 Working as a Team

We split the pipeline into modules — one sub-team owned pre-processing and SPA, another owned core feature extraction, a third owned the extended transforms, and we all collaborated on the selection-and-classification experiments. Keeping clean interfaces between modules (a feature matrix in, a result out) let us work in parallel and swap components without breaking each other's code. The weekly integration sessions, where we had to explain our modules to teammates from different sub-teams, were where most of our shared understanding actually formed.

10. Conclusion

In this project we built, ran, and analysed a complete acoustic fault-diagnosis system for reciprocating air compressors, implementing every stage ourselves in Python: EMD-based sensitive position analysis, a four-step pre-processing routine, multi-domain feature extraction yielding 286 base features (629 in an extended set), six feature-selection methods, and multiclass classification across five models and three decomposition strategies. We backed every major claim with a dedicated, documented experiment rather than a single headline number.

Our best validated configuration — mRMR feature selection, 300 features from the 629-feature extended set, DDAG SVM, $k = 3$ cross-validation — reached **99.44%** accuracy, with the simpler 286-feature base-set equivalent reaching **99.42%**, a difference small enough to be practically irrelevant. This reproduces and validates the central claim of Verma et al. (2016): a *single*, well-placed microphone, paired with the right signal processing and a well-chosen feature subset, is sufficient for highly accurate compressor fault diagnosis. Concretely, our four experiments showed that the Wavelet Packet Transform carries by far the most fault information of any domain we tested; that mRMR is consistently the strongest feature selector, especially at small feature budgets, while PCA can actually *lose* accuracy once too many components are retained; that DDAG matches OAO and OAA in accuracy to within about a percentage point while training 3–5$\times$ faster, making it the pragmatic default; and that tripling the feature vocabulary from 286 to 629 features buys at most 0.02 percentage points of accuracy, meaning the extra transforms are not worth their computational cost for this problem.

More than the numbers, the project gave us a hands-on appreciation of how signal processing and machine learning combine to solve a real industrial problem — and a concrete demonstration that a thoughtfully engineered classical pipeline, with a carefully chosen and appropriately sized feature set, can match a deep network at a tiny fraction of the cost. The model is only ever as good as the features it is built on *and* the selector that chooses among them — and that, more than any single algorithm or feature count, is the lesson we will carry forward.

11. References

[1] N. K. Verma, R. K. Sevakula, S. Dixit, and A. Salour, "Intelligent Condition Based Monitoring Using Acoustic Signals for Air Compressors," *IEEE Transactions on Reliability*, vol. 65, no. 1, pp. 291–309, Mar. 2016.

[2] IIT Kanpur IDEA Lab, *Air Compressor Acoustic Dataset.* [Online]. Available: `https://www.iitk.ac.in/idea/datasets/`

[3] N. E. Huang et al., "The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis," *Proc. R. Soc. London A*, vol. 454, no. 1971, pp. 903–995, 1998.

[4] H. Peng, F. Long, and C. Ding, "Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 8, pp. 1226–1238, 2005.

[5] P. A. Estevez, M. Tesmer, C. A. Perez, and J. M. Zurada, "Normalized mutual information feature selection," *IEEE Trans. Neural Netw.*, vol. 20, no. 2, pp. 189–201, 2009.

[6] R. Battiti, "Using mutual information for selecting features in supervised neural net learning," *IEEE Trans. Neural Netw.*, vol. 5, no. 4, pp. 537–550, 1994.

[7] S. K. Goumas, M. E. Zervakis, and G. S. Stavrakakis, "Classification of washing machines vibration signals using discrete wavelet analysis for feature extraction," *IEEE Trans. Instrum. Meas.*, vol. 51, no. 3, pp. 497–508, 2002.

[8] C. C. Chang and C. J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, 2011.

[9] A. C. Lorena, A. C. P. L. F. de Carvalho, and J. M. P. Gama, "A review on the combination of binary classifiers in multiclass problems," *Artif. Intell. Rev.*, vol. 30, no. 1–4, pp. 19–37, 2008.

[10] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[12] J. M. O'Toole and B. Boashash, "Fast and memory-efficient algorithms for computing quadratic time–frequency distributions," *Applied and Computational Harmonic Analysis*, vol. 35, no. 2, pp. 350–358, 2013.